

Entrega 2 — testes e evidências

Disciplina: Infraestrutura Cloud — UniAmérica **Integrantes:** Luis Felipe Coradi · Rubens Prado · Guilherme Royer
Repositório: <https://github.com/LuisCoradi/aula-uniamerica-infraestrutura-cloud> **Aplicação:** <https://app.trabaidafacul.site> · **API:** <https://api.trabaidafacul.site> **Data dos testes:** 18 de setembro de 2026 · Fuso: America/Sao_Paulo

Todos os cenários foram executados no ambiente real, com dados reais. Nenhum valor apresentado aqui é fictício. Cada cenário traz o que foi feito, o horário, o período consultado no painel, o que foi observado e a conclusão.

Cenário 1 — Uso normal pelo domínio

Objetivo. Demonstrar que a aplicação funciona pelo domínio configurado e que uma operação do usuário percorre todo o caminho até o banco de dados.

Execução. 18/09/2026, por volta das 06:42. Acesso a <https://app.trabaidafacul.site>, criação de tarefas, marcação de tarefas como concluídas e exclusão de tarefa.

Período consultado no painel. Últimos 15 minutos.

Evidências.

- *Figura 1.1* — Aplicação aberta em app.trabaidafacul.site, com a lista de tarefas carregada, itens criados e itens concluídos (riscados). No rodapé, a aplicação indica a API em uso: <https://api.trabaidafacul.site>.
- *Figura 1.2* — Dashboard no mesmo horário. O painel 4 (Uso da aplicação) registra as operações `criar`, `concluir` e `excluir` no minuto da ação. O painel 5 (Banco de dados) registra, no mesmo minuto, as consultas correspondentes no D1: `inserir_tarefa`, `alternar_tarefa` e `excluir_tarefa`, com durações na faixa de 130 ms a 175 ms.

Observado. A ação feita na interface às 06:42 aparece como evento de negócio e como consulta ao banco no mesmo minuto. Os pontos regulares de um em um minuto ao longo de todo o gráfico correspondem à checagem automática do Worker monitor, o que mostra que a coleta é contínua e independe de haver usuário.

Conclusão. O acesso ocorre pelo domínio configurado e a operação percorre front-end, proxy reverso, back-end e banco, com registro observável em cada etapa.

Cenário 2 — Volume de requisições

Objetivo. Gerar amostras suficientes para que os indicadores estatísticos (percentil 95 e mediana) tenham significado, e observar a distribuição entre as origens do front-end.

Execução. 18/09/2026, entre 06:47 e 06:50. Sessenta requisições alternadas a <https://api.trabaidafacul.site/todos> e <https://app.trabaidafacul.site/>, disparadas em sequência.

Período consultado no painel. Últimos 15 minutos.

Evidências.

- *Figura 2.1* — Painel 2 (Latência por rota). O bloco denso de amostras corresponde ao período do teste. Valores por rota: `/health` com p95 de 140 ms, `/todos` com 127 ms e `/todos/:id` com 171 ms.
- *Figura 2.2* — Painel 6 (Redundância do front-end) no mesmo intervalo, registrando as requisições atendidas e a origem responsável por cada uma.

Observado. Com volume, a série de latência passa de pontos isolados a uma linha contínua, e o percentil 95 fica estável na faixa de 130 ms a 180 ms. O painel 6 acompanha o aumento de requisições no mesmo intervalo.

Conclusão. Os indicadores de desempenho respondem ao volume real de tráfego e se mantêm consistentes sob carga contínua.

Cenário 3 — Regras de acesso do proxy reverso

Objetivo. Demonstrar que as regras de acesso aplicadas no proxy funcionam e que o painel distingue bloqueio proposital de falha da aplicação.

Execução. 18/09/2026, entre 06:55 e 07:03. Três tipos de requisição, feitas com `curl` diretamente contra api.trabaidafacul.site:

1. Requisição com cabeçalho `Origin: https://site-malicioso.com`

2. Requisição à rota `/admin`, fora da lista de rotas permitidas
3. Requisição com método `PUT`, fora da lista de métodos permitidos

Período consultado no painel. Últimos 15 minutos.

Evidências.

- **Figura 3.1** — Resposta `HTTP/1.1 403 Forbidden` para a origem não autorizada, com corpo `{"message":"Origin https://site-malicioso.com nao autorizada", "bloqueado_por":"proxy-reverso"}`. Na resposta aparecem também os cabeçalhos de segurança aplicados: `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, além de `X-Proxy: cloudflare-worker-reverse-proxy` e do identificador `X-Req-Id`.
- **Figura 3.2** — Resposta `HTTP/1.1 403 Forbidden` para a rota `/admin`.
- **Figura 3.3** — Resposta `HTTP/1.1 405 Method Not Allowed` para o método `PUT`, com corpo `{"message":"Metodo PUT nao permitido", "bloqueado_por":"proxy-reverso"}`.
- **Figura 3.4** — Pannel 3 (Erros e bloqueios) no horário das requisições, com as três séries separadas: `origem_nao_autorizada`, `rota_nao_permitida` e `metodo_nao_permitido`.

Observado. Cada tentativa foi recusada pelo proxy com o código correto e com a identificação de quem bloqueou. As três aparecem no painel como séries distintas, classificadas pelo motivo, enquanto a série de erros de servidor permanece sem ocorrências.

Conclusão. As regras de acesso funcionam conforme especificado, e o painel permite diferenciar, à primeira vista, um bloqueio proposital de uma falha real da aplicação.

Cenário 4 — Falha controlada e failover do front-end

Objetivo. Provocar a indisponibilidade de uma das origens do front-end e demonstrar que a redundância entra em ação e é observável no painel.

Execução. 18/09/2026, a partir das 18:25. A variável `ORIGIN_1` do proxy reverso foi alterada de `trabaidafacul.pages.dev` para `origem-fora-do-ar.pages.dev`, um host inexistente, e o Worker foi republicado. Em seguida foram geradas requisições ao domínio. Ao final do cenário, a configuração original foi restaurada e o Worker republicado.

Período consultado no painel. Últimos 15 minutos.

Evidências.

- **Figura 4.1** — Pannel 6 (Redundância do front-end) às 18:28. Três séries presentes: `QUEDA origem-fora-do-ar.pages.dev`, registrando a origem que não respondeu; `requisicoes com failover`, contabilizando as requisições reencaminhadas; e `atendido por trabaidafacul-b.pages.dev`, mostrando que a segunda origem assumiu o atendimento.
- **Figura 4.2** — Saída do `curl` às 18:32 contra `https://app.trabaidafacul.site/`, com os cabeçalhos `X-Failover: sim` e `X-Served-By: trabaidafacul-b.pages.dev`.

Observado. Com a origem preferida indisponível, o proxy detectou a falha, reencaminhou a requisição para a segunda origem e devolveu a página normalmente ao usuário. O painel registrou o evento de queda, a origem que assumiu e a quantidade de requisições que passaram por failover, todos no mesmo intervalo.

Conclusão. A redundância do front-end funciona na prática: a indisponibilidade de uma origem não interrompe o atendimento, e o evento fica documentado no painel com horário exato. Após o teste, o ambiente foi restaurado à configuração original.

Cenário 5 — Caminho de um evento, da aplicação ao painel

Objetivo. Demonstrar o percurso completo de uma requisição: a ação executada, o registro gerado na fonte de dados e sua apresentação na ferramenta de consulta.

Execução. 18/09/2026, às 18:36. Requisição a `https://api.trabaidafacul.site/todos`, com captura do identificador de correlação devolvido na resposta. Em seguida, consulta desse identificador no Grafana, às 18:41.

Período consultado no painel. Últimos 15 minutos, com filtro pelo identificador da requisição.

Evidências.

- **Figura 5.1** — Saída do `curl` mostrando o cabeçalho `X-Req-Id: fb11824c-157a-49e0-880e-2c6f67b8be23`.
- **Figura 5.2** — Consulta no Grafana filtrando por esse mesmo identificador, retornando as três linhas geradas por aquela única requisição:

1. `servico: proxy, evento requisicao_api, método GET, rota /todos, status 200, duração 164 ms, upstream: service-binding, trafego: usuario, país BR`
2. `servico: api, evento consulta_banco, operação listar_tarefas, banco trabaidafacul-db, ok: true, duração 136 ms`
3. `servico: api, evento operacao_negocio, operação listar`

O painel lateral do Grafana mostra os campos extraídos e indexados a partir do JSON: `aplicacao`, `ambiente`, `servico`, `req_id`, `rota`, `status`, `duracao_ms`, `metodo`, `evento`, `trafego`, entre outros.

Observado. O identificador devolvido ao cliente na resposta HTTP é o mesmo que aparece nos registros do proxy, do back-end e da consulta ao banco. Os três eventos têm o mesmo carimbo de tempo, com diferença de milissegundos entre eles, e permitem reconstruir a requisição inteira.

Conclusão. O caminho entre o evento ocorrido na aplicação, seu registro na fonte de dados e sua apresentação na ferramenta de consulta está demonstrado de ponta a ponta, com rastreabilidade por requisição individual.

Cenário 6 — Correspondência entre o diagrama e o ambiente

Objetivo. Demonstrar que os recursos representados no diagrama técnico correspondem aos recursos efetivamente existentes na conta da Cloudflare.

Execução. 18/09/2026, entre 18:54 e 18:57. Consulta ao painel administrativo da Cloudflare.

Evidências.

- *Figura 6.1* — Lista de Workers e Pages da conta, mostrando os cinco recursos representados no diagrama:
 - `trabaidafacul-proxy`, com `app.trabaidafacul.site` e mais uma rota
 - `trabaidafacul-monitor`, sem rotas ativas
 - `trabaidafacul-api`, sem rotas ativas
 - Projetos de páginas `trabaidafacul` (`trabaidafacul.pages.dev`) e `trabaidafacul-b` (`trabaidafacul-b.pages.dev`), as duas origens do front-end

O painel de uso da conta no mesmo print registra 10,47 mil requisições e 18,6 mil eventos de observabilidade no período de 1 a 18 de setembro.

- *Figura 6.2* — Aba de domínios do Worker `trabaidafacul-proxy`, com os dois Custom Domains de produção, `app.trabaidafacul.site` e `api.trabaidafacul.site`, ambos na zona `trabaidafacul.site`, e as URLs padrão `workers.dev` desativadas.
- *Figura 6.3* — Aba de eventos de gatilho do Worker `trabaidafacul-monitor`, mostrando o Cron Trigger configurado para executar a cada minuto, com a próxima execução agendada.

Observado. Os nomes de recursos que aparecem no diagrama são os mesmos da conta: três Workers, dois projetos de páginas, dois hostnames de produção e o agendamento de um minuto da sonda. O proxy é o único recurso com domínios associados; o back-end e o monitor aparecem sem rotas ativas, conforme representado na zona privada do diagrama.

Conclusão. O diagrama técnico corresponde ao ambiente real em execução.

Resumo dos cenários

Cenário	O que demonstra	Painéis validados
1	Uso normal pelo domínio, do front-end ao banco	2, 4, 5
2	Comportamento dos indicadores sob volume	2, 6
3	Regras de acesso e distinção entre bloqueio e falha	3
4	Falha controlada e failover do front-end	1, 1b, 2, 6
5	Caminho do evento até o painel, por requisição	todos
6	Correspondência entre diagrama e ambiente	—

Ao final dos testes, a configuração original do proxy reverso foi restaurada e o ambiente permaneceu em operação normal, com a coleta de dados seguindo ativa.